

IF2211 STRATEGI ALGORITMA

Laporan Tugas Kecil 1

Semester II – 2025/2026



PENYELESAIAN PERMAINAN QUEENS LINKEDIN DENGAN ALGORITMA BRUTE FORCE

Disusun oleh:

Gabriella Botimada Lubis 13524006

INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

DAFTAR ISI.....	2
BAB 1 DESKRIPSI PERSOALAN.....	3
BAB 2 ALGORITMA BRUTE FORCE.....	4
BAB 3 IMPLEMENTASI.....	5
BAB 4 UJI KASUS.....	8
Test Case 1.....	8
Test Case 2.....	9
Test Case 3.....	11
Test Case 4.....	12
Test Case 5.....	13
Test Case 6.....	15
Test Case 7.....	15
Test Case 8.....	16
LAMPIRAN.....	17

BAB 1

DESKRIPSI PERSOALAN

Queens merupakan sebuah permainan logika yang bertujuan untuk menempatkan queen pada sebuah papan persegi berwarna sehingga terdapat tepat satu queen pada tiap baris, kolom, dan daerah warna, serta tidak ada queen yang saling bersebelahan termasuk secara diagonal. Melalui tugas kecil ini, dibuat sebuah program untuk menemukan sebuah solusi penempatan queen pada papan yang diberikan. Secara lebih rinci, program memiliki spesifikasi seperti berikut.

- Menerima input berupa file .txt
- Membaca file input yang berisi konfigurasi awal papan yang akan diselesaikan, kemudian memvalidasi isinya (papan harus dilambangkan oleh alfabet yang tiap hurufnya melambangkan daerah warna, papan harus berbentuk persegi, dan pada papan berukuran $n \times n$ harus terbagi menjadi n daerah warna)
- Menemukan solusi penempatan queen secara Brute Force dan menampilkan *live update* prosesnya
- Menampilkan solusi papan beserta waktu pencarian dan banyak kasus yang ditinjau, atau menampilkan bahwa tidak ada solusi
- Menyimpan solusi dalam file .txt atau .png

BAB 2

ALGORITMA BRUTE FORCE

Algoritma Brute force merupakan algoritma yang menyelesaikan persoalan secara straightforward, yaitu dengan sederhana, langsung, cara yang jelas, dan langkah-langkah yang mudah dipahami. Dalam program ini, algoritma Brute force diimplementasikan dengan mencoba semua kemungkinan penempatan n buah queens pada papan berukuran $n \times n$ yang memiliki n daerah warna. Kemudian untuk setiap kemungkinan, program memeriksa apakah konfigurasi penempatan tersebut memenuhi aturan permainan. Langkah-langkah algoritma antara lain:

1. Percobaan pertama menempatkan semua queen pada kolom pertama.
2. Dilakukan pengecekan apakah ada dua queen pada kolom yang sama, daerah warna yang sama, atau bersebelahan secara diagonal. Jika ada satu kondisi dilanggar, program langsung beralih ke konfigurasi selanjutnya.
3. Untuk kasus-kasus setelahnya, queen pada baris terakhir digeser maju 1 kotak, jika sudah mencapai ujung, queen tersebut kembali ke kolom pertama dan queen pada baris sebelumnya maju 1 kotak. Percobaan terus dilakukan hingga semua queen telah berada di kolom terakhir.
4. Untuk setiap konfigurasi, dilakukan pengecekan seperti langkah 2. Jika tidak ada yang dilanggar, maka solusi ditemukan dan pencarian dihentikan.

BAB 3

IMPLEMENTASI

Penempatan queen direpresentasikan oleh sebuah array queens berukuran n , di mana `queens[i]` melambangkan kolom tempat queen pada baris i . Misalnya jika terdapat `queens = [1, 3, 0, 2]`, maka berarti queen ditempatkan di baris 0 kolom 1, baris 1 kolom 3, baris 2 kolom 0, dan baris 3 kolom 2. Karena setiap baris selalu memiliki 1 queen, maka total kemungkinan adalah n^n .

```
n = len(board)
queens = [0]*(n-1)
queens.append(-1)
```

Inisialisasi variabel `iter` dan `start` dilakukan untuk menyimpan banyak iterasi dan waktu mulai pencarian

```
iter = 0
start = time.perf_counter()
```

Fungsi `iterate` digunakan sebagai *helper* untuk menghasilkan konfigurasi queen berikutnya menggunakan metode enumerasi. Fungsi ini menerima parameter `queens` sebagai konfigurasi queen sebelumnya, `n` sebagai indeks baris yang sedang di-*increment*, dan `nBoard` yang menunjukkan ukuran papan. Setiap fungsi `iterate` dipanggil, nilai elemen terakhir dari array `queens` ditambah 1. Jika nilainya menjadi sama dengan `nBoard`, nilai elemen tersebut dikembalikan menjadi 0, dan nilai elemen sebelumnya ditambah 1 melalui pemanggilan rekursif. Fungsi ini akan mengembalikan `True` jika terdapat konfigurasi yang memungkinkan dan `False` jika setiap elemen telah *overflow*.

```
def iterate (queens: list of int, n: int, nBoard: int):
    if (n == 0):
        return False
    queens[n-1] += 1
    if queens[n-1] == nBoard:
        queens[n-1] = 0
        return iterate (queens, n-1, nBoard)
    return True
```

Pada fungsi solve() yang merupakan algoritma inti program, fungsi iterate dipanggil selama masih ada kemungkinan konfigurasi. Jika sudah habis, maka program akan memunculkan pesan bahwa tidak ada solusi untuk papan yang dimasukkan.

```
while True:
    if not iterate (queens, n, n):
        messagebox.showinfo("NO SOLUTION", "This board has no
        solution.")
        return
```

Setelah sebuah kemungkinan konfigurasi disusun, dilakukan pengecekan apakah konfigurasi tersebut memenuhi syarat permainan. Pengecekan selalu dimulai dengan membuat array kosong cols dan colors yang menyimpan kolom dan warna apa saja yang ditempati tiap queens. Jika ada queen yang menempati kolom atau warna yang sudah ditempati queen sebelumnya, pengecekan langsung dihentikan dan pencarian kembali dilanjutkan. Selain itu, dipastikan pula tidak ada queen yang bersebelahan secara diagonal dengan memastikan queen pada baris i tidak boleh berada tepat di sebelah queen pada baris i+1. Jika sebuah konfigurasi queen lolos semua pengecekan, pencarian berhenti dan program akan memunculkan solusi, waktu pencarian, dan banyak kasus yang ditinjau untuk menemukan solusi.

```
solved = True

cols = []
colors = []
for i in range (n):
    column_i = queens[i]
    color_i = board[i][column_i]
    if column_i in cols:
        solved = False
        break
    if color_i in colors:
        solved = False
        break
    if i<n-1 and abs(column_i - queens[i+1]) == 1:
        solved = False
        break
```

```
        cols.append(column_i)
        colors.append(color_i)

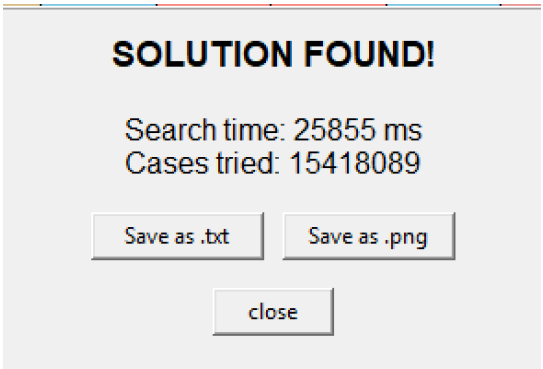
    if solved:
        end = time.perf_counter()
        result_popup(home, end-start, iter, board, queens, n)
        return
```

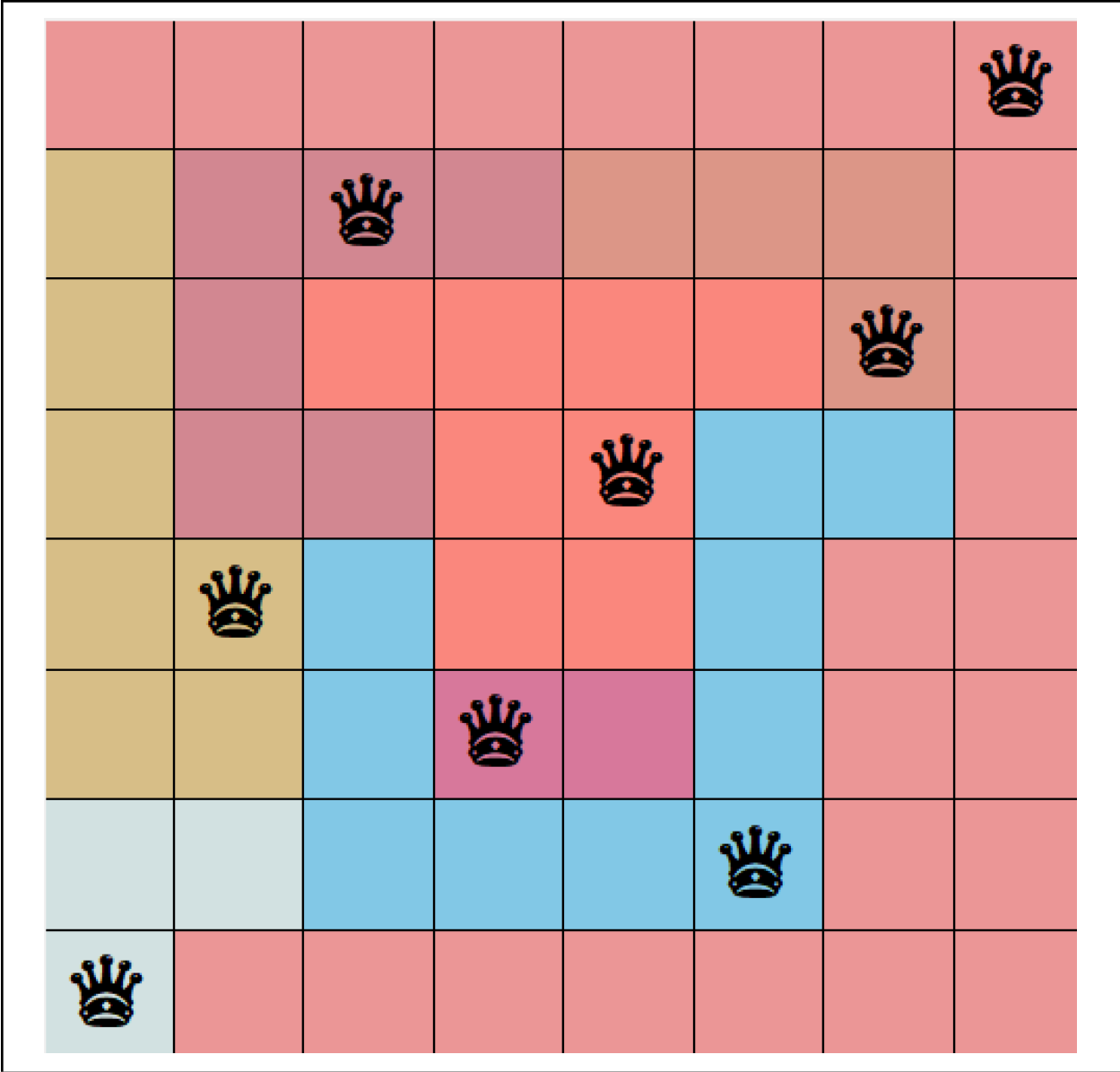
Untuk menampilkan *live update*, program diatur agar menampilkan konfigurasi queen terbaru tiap 1 detik agar tidak terlalu berat dan lama jika harus menampilkan tiap iterasi.

```
    now = time.perf_counter()
    if (now-last >= 1):
        result_img(board, queens, n)
        home.update()
        last = now
```

BAB 4
UJI KASUS

Test Case 1

Input
AAAAAAAAA BGGGHHHA BGFFFFHA BGGFFDDA BBDFFDAA BBDEEDAA CCDDDDAA CAAAAAAAAA
Output
 SOLUTION FOUND! Search time: 25855 ms Cases tried: 15418089 Save as .txt Save as .png close



Test Case 2

Input
AAABBCCCD
ABBBBCECD
ABBBDCED
AAABDCCCD
BBBBDDDDD
FGGGDDHDD
FGIGDDHDD
FGIGDDHDD

FGGGDDHHH

Output

SOLUTION FOUND!

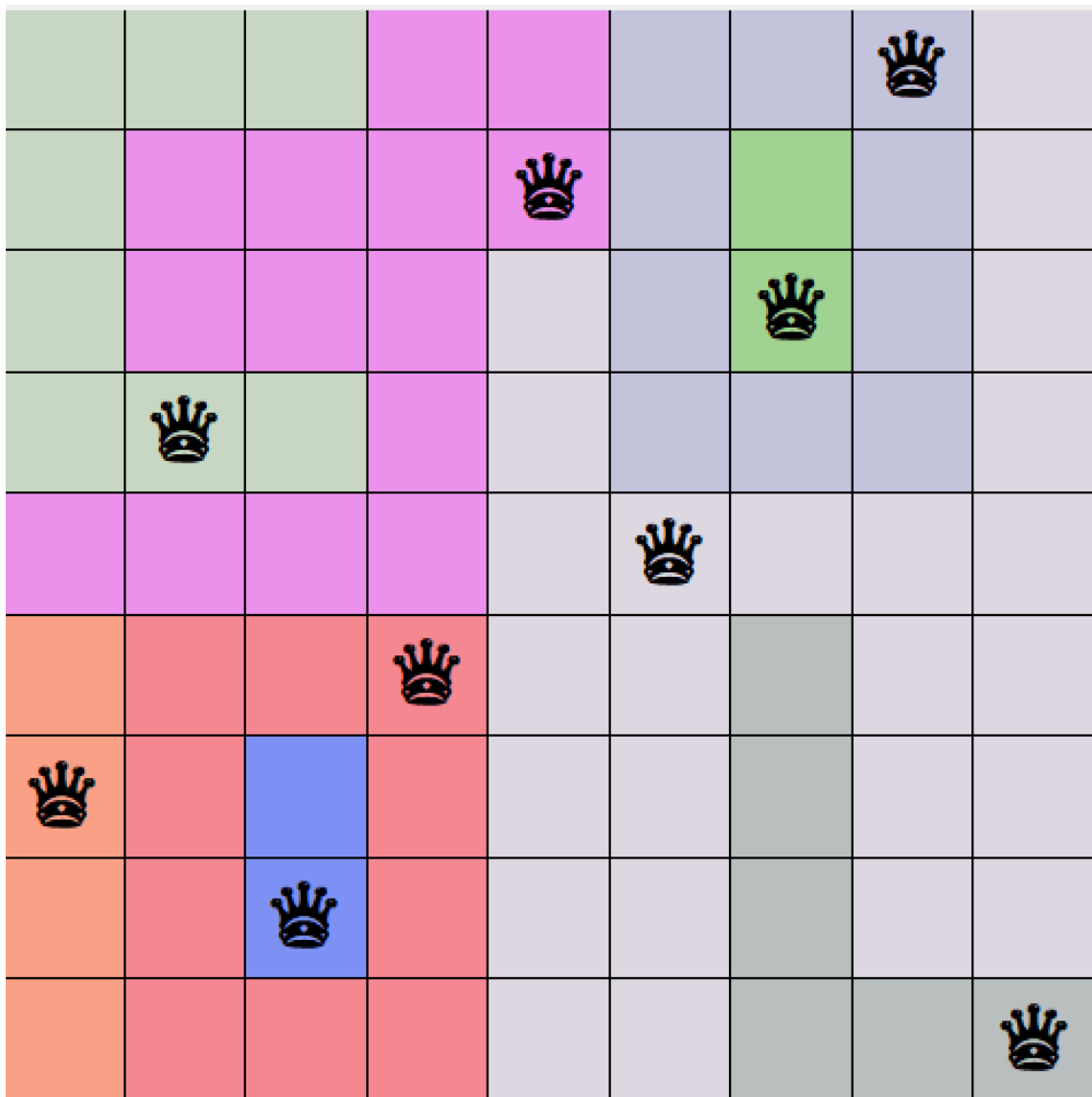
Search time: 509457 ms

Cases tried: 323741637

Save as .txt

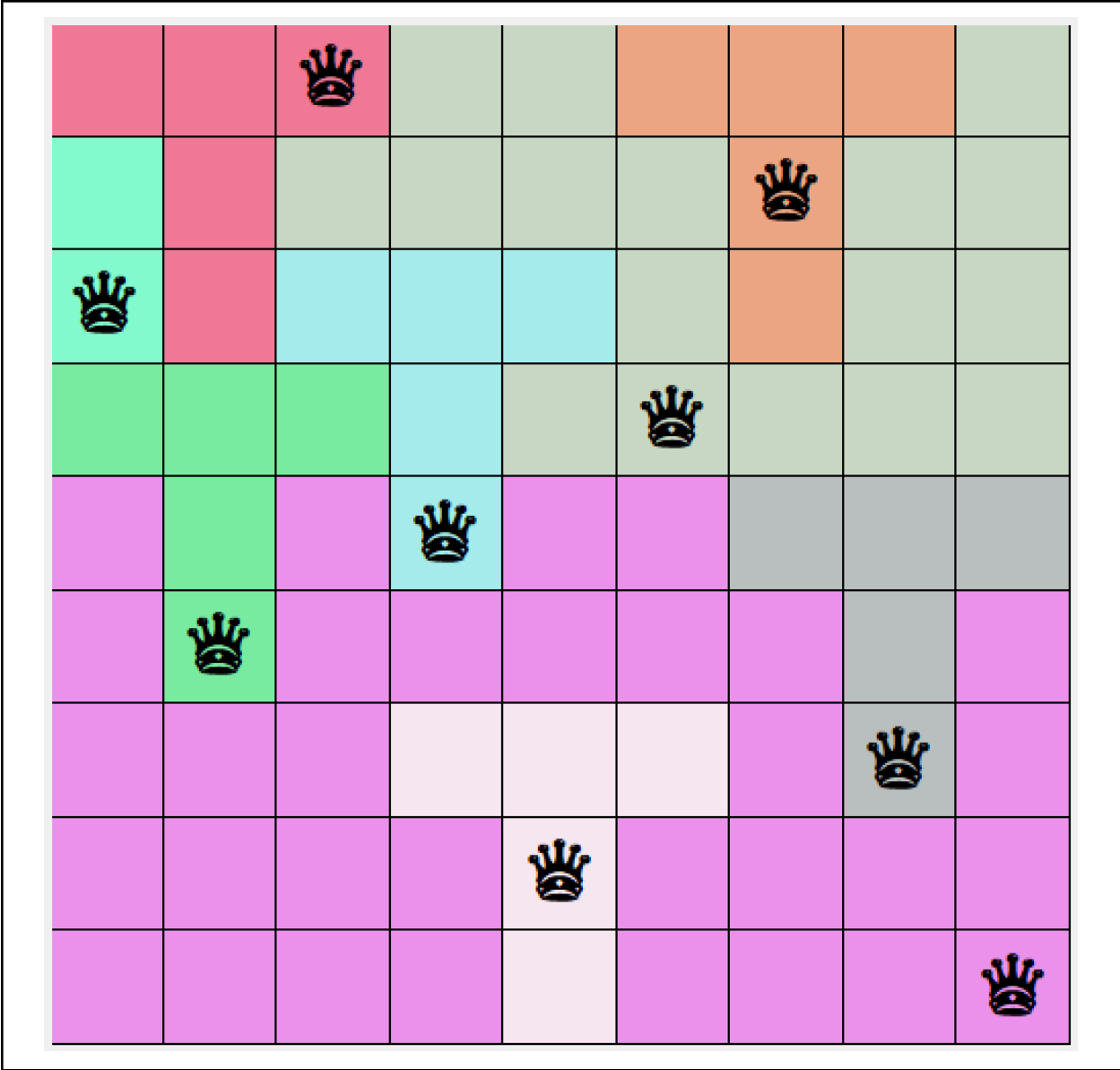
Save as .png

close



Test Case 3

Input
KKKAAOOOA MKAAAAOAA MKUUUAOAA PPUAAAAA BPBUBBHHH BPBBBBBHB BBBSSSBHB BBBBSBBBB BBBBSBBBB
Output
SOLUTION FOUND! Search time: 193587 ms Cases tried: 115107525 Save as .txt Save as .png close



Test Case 4

Input
AAAAA BGGGHHHA BGFFFFHA BGGFDDAA BBDFDDAA BBDEEDAA CCDDDDAA CAAAAAA

Output

NO SOLUTION

 This board has no solution.

OK

Test Case 5

Input	
	A A A A A A B A A A A A B B A C C C C C B A C D D D C E A C C C C C E A A A F G E E

A A F F G G G

Output

SOLUTION FOUND!

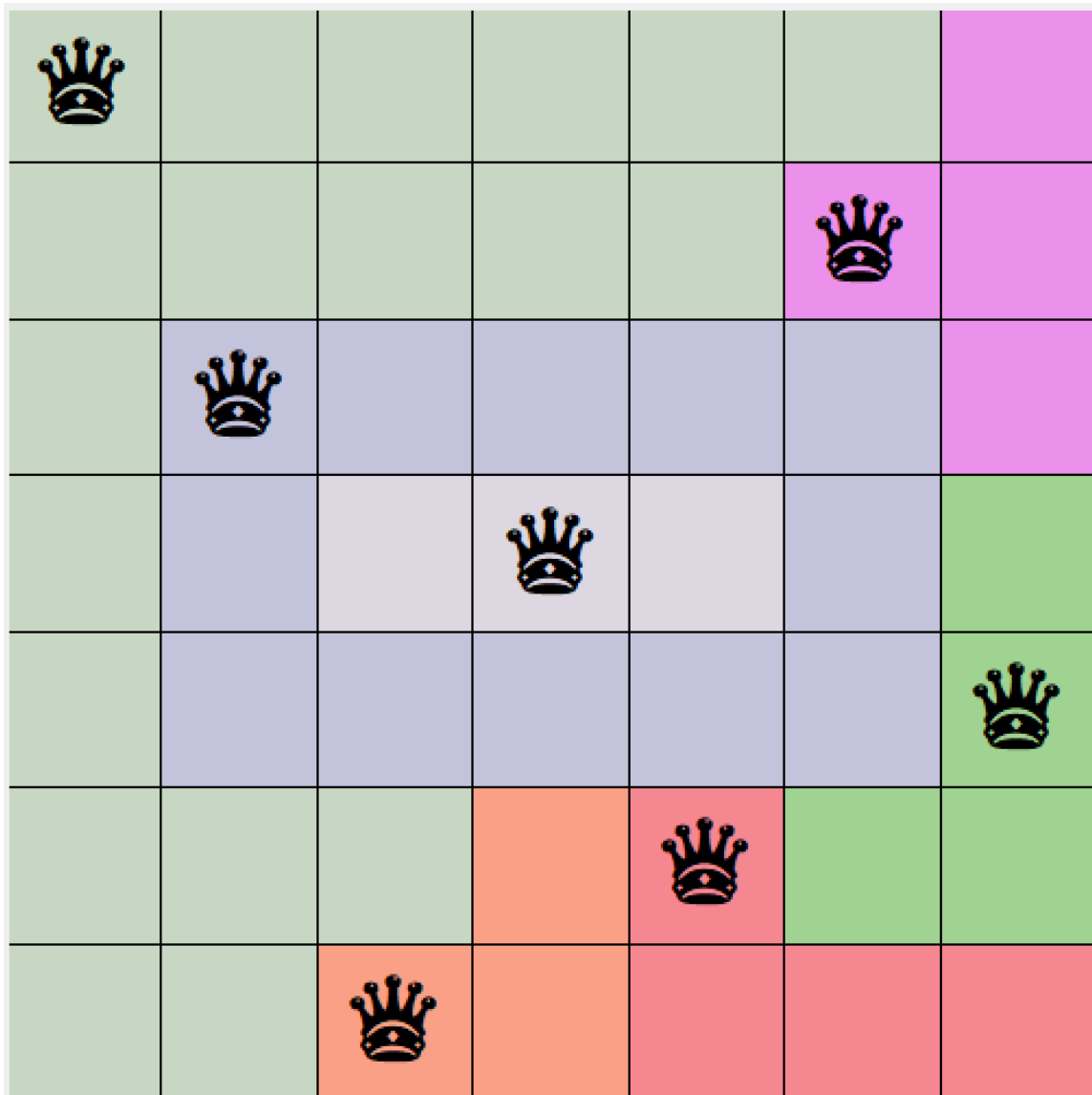
Search time: 135 ms

Cases tried: 87790

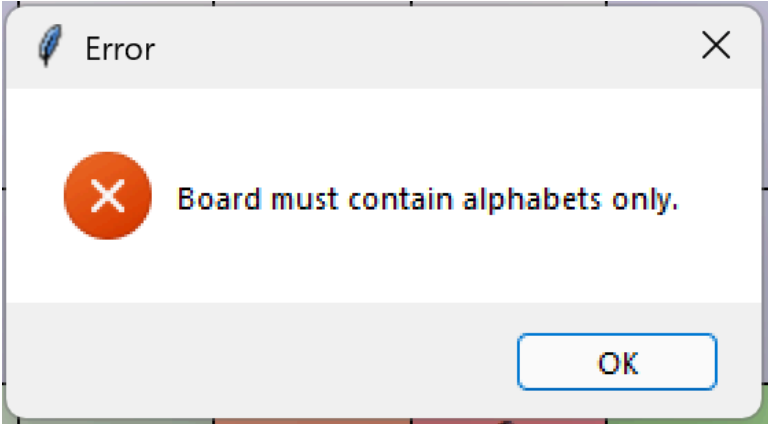
Save as .txt

Save as .png

close

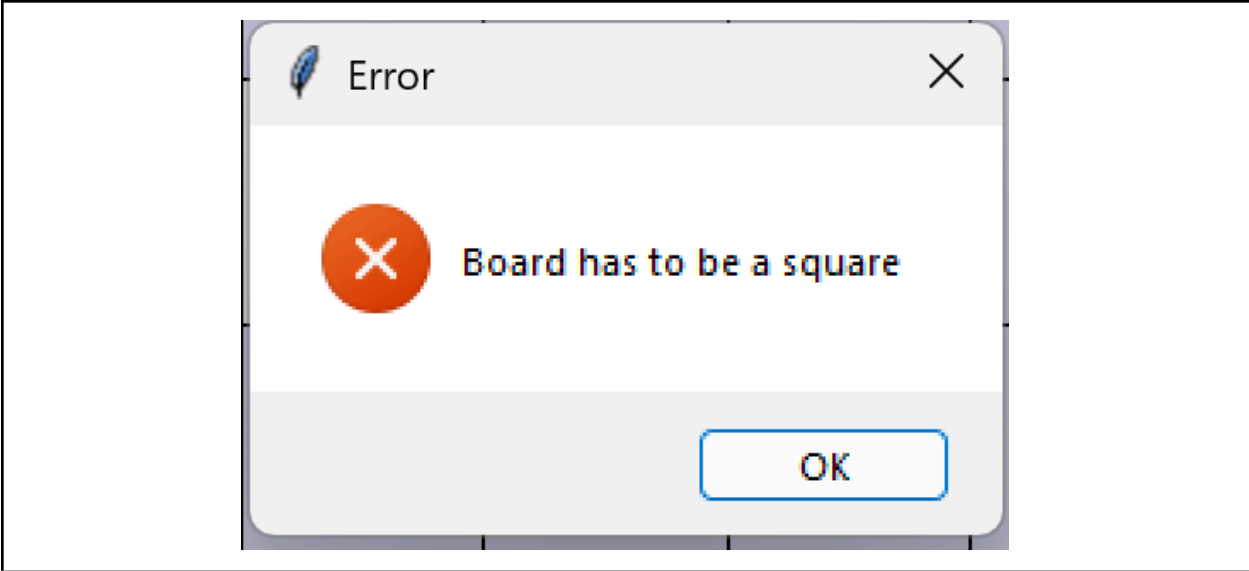


Test Case 6

Input
AA6BBCCCD ABBB'CECD ABBBDCEDD AAABDCCCD BBBB.DDDD FGGGDDHDD FG9GDDHDD FGIGDD"DD FGGGDDHHH
Output
 An error dialog box with a title bar containing a feather icon, the word "Error", and a close button (X). The main area contains a red circle with a white X icon and the text "Board must contain alphabets only." Below this is an "OK" button.

Test Case 7

Input
KKKAAOOOA MKAAAAOAA MKUUUAOAA PPPUAAAAA BPBUBBHHH BPBBBBBHB BBBSSBHB BBBBSB BBB
Output



Test Case 8

Input
AAABBCCCD ABBBBCECD ABBBDCEDC AAABDCCCD BBBBDDDDD FGGGDDHDD FGGGDDHDD FGGGDDHDD FGGGDDHHH
Output
An error dialog box with a title bar containing a feather icon, the word "Error", and a close button. The main area features a red circle with a white 'X' icon followed by the text "Regions count must be as many as the board's dimension.". At the bottom right is an "OK" button.

LAMPIRAN

Pranala ke repository: https://github.com/gabriellaalubis/Tucil1_13524006

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Program dapat menyimpan solusi dalam bentuk file gambar	✓	

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Gabriella Botimada Lubis